

保険約款ドラフト作成アプリケーション 商用 利用向け技術選定レポート

エグゼクティブサマリー

本レポートは、商用利用を前提とした保険約款ドラフト作成アプリケーションの技術選定について、VSCode/Clineのライセンス制約を踏まえた独自開発可能な代替技術スタックを提案するものです。

主要な調査結果

1. **VSCode/Clineの商用利用制約**
2. VSCode製品：商標権により「Visual Studio Code」名称の商用利用は制限
3. Clineソースコード：Apache 2.0により商用利用は可能だが独自ブランディングが必要
4. **推奨技術スタック**
5. **最優先推奨**：Theia IDE Platform + 独自カスタマイズ
6. **代替案1**：Tauri + Monaco Editor + React
7. **代替案2**：Webアプリケーション（Monaco Editor + Node.js）
8. **開発期間・コスト見積もり**
9. Theia IDEベース：4-6ヶ月、開発コスト低-中
10. Tauri + Monaco：8-10ヶ月、開発コスト中-高
11. 完全独自開発：12-18ヶ月、開発コスト高

1. 背景と目的

1.1 プロジェクト要件

- ・ 商用利用を前提とした保険約款ドラフト作成アプリケーション
- ・ VSCode + Cline拡張機能のような生成AIとの対話型インターフェース
- ・ エディターでの手動編集とAIリクエストの両方に対応
- ・ 過去の約款ドキュメントを参考にした記載の踏襲機能
- ・ UI上でのVSCode/Cline名称表示の回避

1.2 調査目的

- ・ VSCode/Clineの商用利用におけるライセンス制約の明確化
- ・ 独自ブランディング可能な代替技術スタックの特定
- ・ 商用利用に適したライセンス体系の選定
- ・ 開発期間・コスト・リスクの評価

2. ライセンス分析結果

2.1 VSCodeライセンス状況

VSCode製品ライセンス

- ・ **ライセンス:** Microsoft Software License Terms
- ・ **商用利用:** 可能（開発・テスト・社内利用）
- ・ **制約事項:**
 - ・ 「Visual Studio Code」 名称の商用利用は商標権により制限
 - ・ ソフトウェアの再配布・販売は禁止
 - ・ スタンドアロン製品としての提供は禁止

VSCodeソースコード

- ・ **ライセンス:** MIT License
- ・ **権利:** 商用利用・修正・配布・サブライセンス・販売すべて可能
- ・ **条件:** 著作権表示とライセンス表示の保持のみ
- ・ **商標権:** 「Visual Studio Code」 名称使用は別途制限

2.2 Clineライセンス状況

Clineソースコード

- ・ **ライセンス:** Apache 2.0 License
- ・ **権利:** 商用利用・修正・配布・サブライセンス・特許権許可すべて可能
- ・ **条件:** 著作権表示・ライセンス表示・変更箇所明示・NOTICE保持
- ・ **商標権:** 「Cline」 名称使用に特別な制限なし

2.3 商用利用リスク評価

高リスク要素

- ・ VSCode製品の直接的な再配布・販売
- ・ 「Visual Studio Code」 「VS Code」 名称の商用利用

- ・ VSCode拡張機能としての直接販売

低リスク要素

- ・ VSCodeソースコードベースの独自エディター開発（商標権注意）
- ・ Clineソースコードの改変利用（Apache 2.0準拠）
- ・ 完全独自ブランディングでの提供

3. 代替技術スタック分析

3.1 デスクトップアプリケーション開発技術

Tauri（推奨度：最高）

概要: Rustベースの軽量デスクトップアプリケーションフレームワーク

技術仕様: - 言語: Rust（バックエンド） + Web技術（フロントエンド） - **ライセンス:** MIT/Apache 2.0 - **バンドルサイズ:** 10-20MB（Electronの1/10） - **メモリ使用量:** 50-100MB（Electronの1/5） - **プラットフォーム:** Windows, macOS, Linux, iOS, Android

商用利用評価: -  完全な商用利用許可 -  独自ブランディング可能 -  再配布・販売可能 -  ソースコード保護可能

開発コスト: - 学習コスト: 高（Rust習得必要） - 開発期間: 8-10ヶ月 - 保守コスト: 中

Wails（推奨度：中）

概要: Goベースの軽量デスクトップアプリケーションフレームワーク

技術仕様: - 言語: Go（バックエンド） + Web技術（フロントエンド） - **ライセンス:** MIT - **バンドルサイズ:** 15-30MB - **開発速度:** 高（Goの生産性） - **制約:** 一部機能が開発中（マルチウィンドウ等）

商用利用評価: -  完全な商用利用許可 -  独自ブランディング可能 -  機能制限あり

開発コスト: - 学習コスト: 中（Go習得必要） - 開発期間: 6-8ヶ月 - 保守コスト: 低

3.2 Webベースエディター技術

Monaco Editor（推奨度：最高）

概要: VSCodeのコアエディターをWeb向けに抽出したライブラリ

技術仕様: - **開発元:** Microsoft - **ライセンス:** MIT - **言語サポート:** 100+ プログラミング言語 - **機能:** IntelliSense, シンタックスハイライト, 差分表示, 拡張可能 - **制約:** モバイル非対応

商用利用評価: - ☒ 完全な商用利用許可 (MIT) - ☒ 独自ブランディング可能 - ☒ カスタマイズ自由度最高 - ☒ VSCode互換性

統合コスト: - 学習コスト: 低 - 統合期間: 2-3ヶ月 - カスタマイズ性: 最高

CodeMirror 6 (推奨度: 高)

概要: モジュラー設計の軽量Webエディター

技術仕様: - ライセンス: MIT - 設計: モジュラー・拡張可能 - モバイル: 対応 - パフォーマンス: 高

商用利用評価: - ☒ 完全な商用利用許可 - ☒ 高カスタマイズ性 -  Monaco Editorより機能少

3.3 統合開発環境プラットフォーム

Theia IDE Platform (推奨度: 最高)

概要: Eclipse Foundationによるクラウド・デスクトップ対応IDE開発プラットフォーム

技術仕様: - 開発元: Eclipse Foundation - ライセンス: EPL 2.0 (商用フレンドリー) - アーキテクチャ: TypeScript, Node.js - 拡張性: VSCode拡張機能互換 - AI統合: Theia AI対応

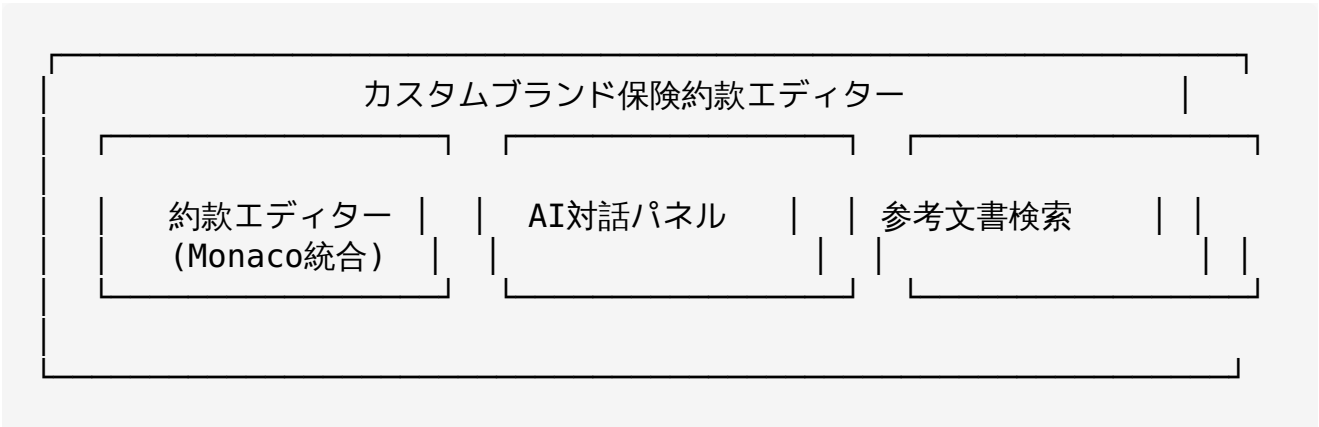
商用利用評価: - ☒ 商用利用フレンドリー (EPL 2.0) - ☒ 独自ブランディング可能 - ☒ 商用サポート利用可能 - ☒ VSCode代替として実績あり

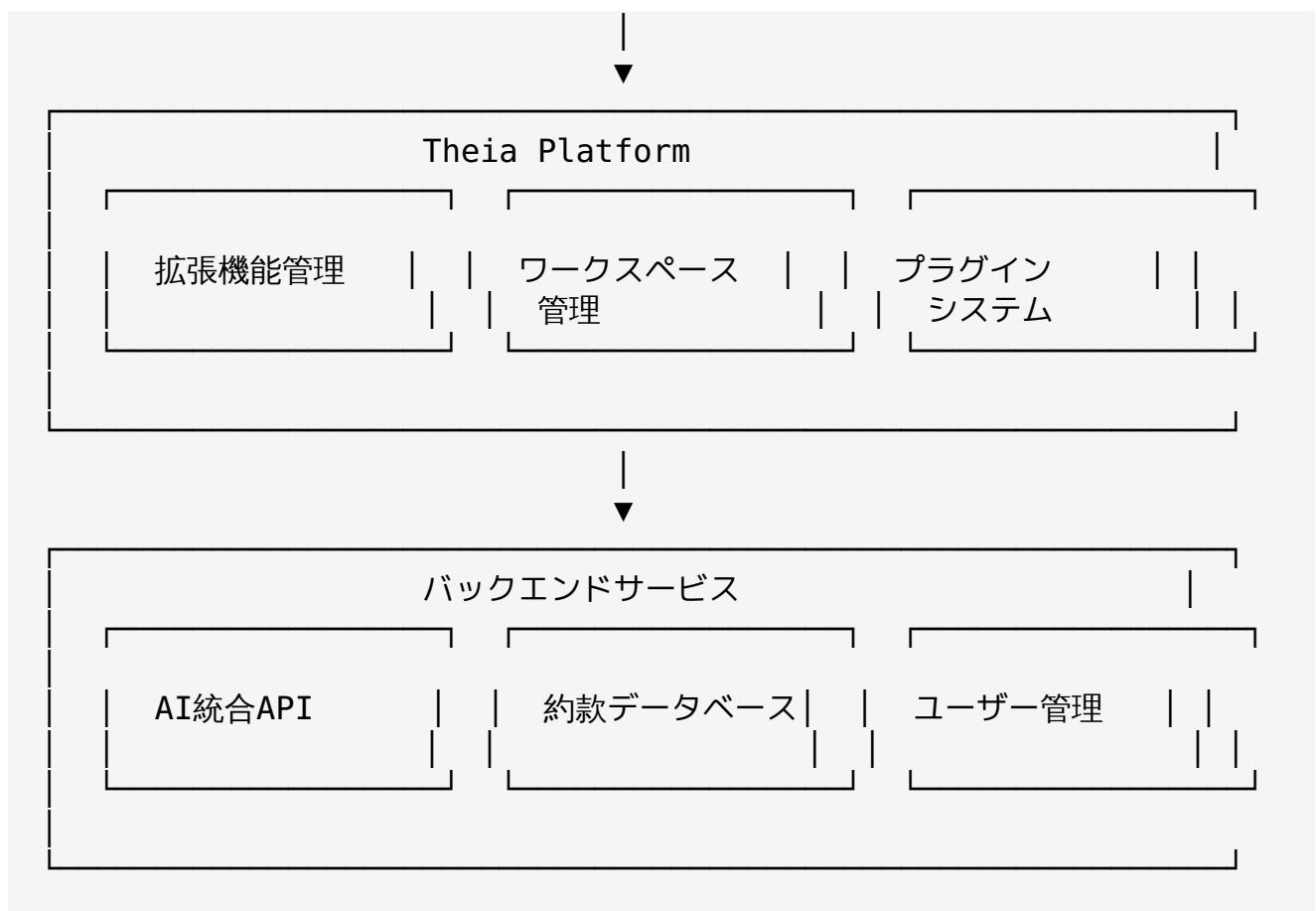
開発コスト: - 学習コスト: 中 (TypeScript) - 開発期間: 4-6ヶ月 - 保守コスト: 低-中 - 商用サポート: 利用可能

4. 推奨技術スタック

4.1 最優先推奨: Theia IDEベース

アーキテクチャ





技術構成

- ベースプラットフォーム: Theia IDE Platform
- エディターコア: Monaco Editor (Theia統合済み)
- フロントエンド: TypeScript + React
- バックエンド: Node.js + Express
- AI統合: OpenAI/Anthropic/Mistral API
- データベース: PostgreSQL + pgvector
- デプロイ: Docker + Kubernetes

開発工程

1. **Phase 1 (1-2ヶ月)** : Theia環境構築・基本カスタマイズ
2. **Phase 2 (2-3ヶ月)** : 約款エディター機能開発
3. **Phase 3 (1-2ヶ月)** : AI統合・RAG機能実装
4. **Phase 4 (1ヶ月)** : テスト・デプロイ・運用準備

利点

- ☒ 開発期間短縮 (既存プラットフォーム活用)
- ☒ VSCode互換性 (拡張機能流用可能)
- ☒ 商用サポート利用可能
- ☒ 実績あるプラットフォーム

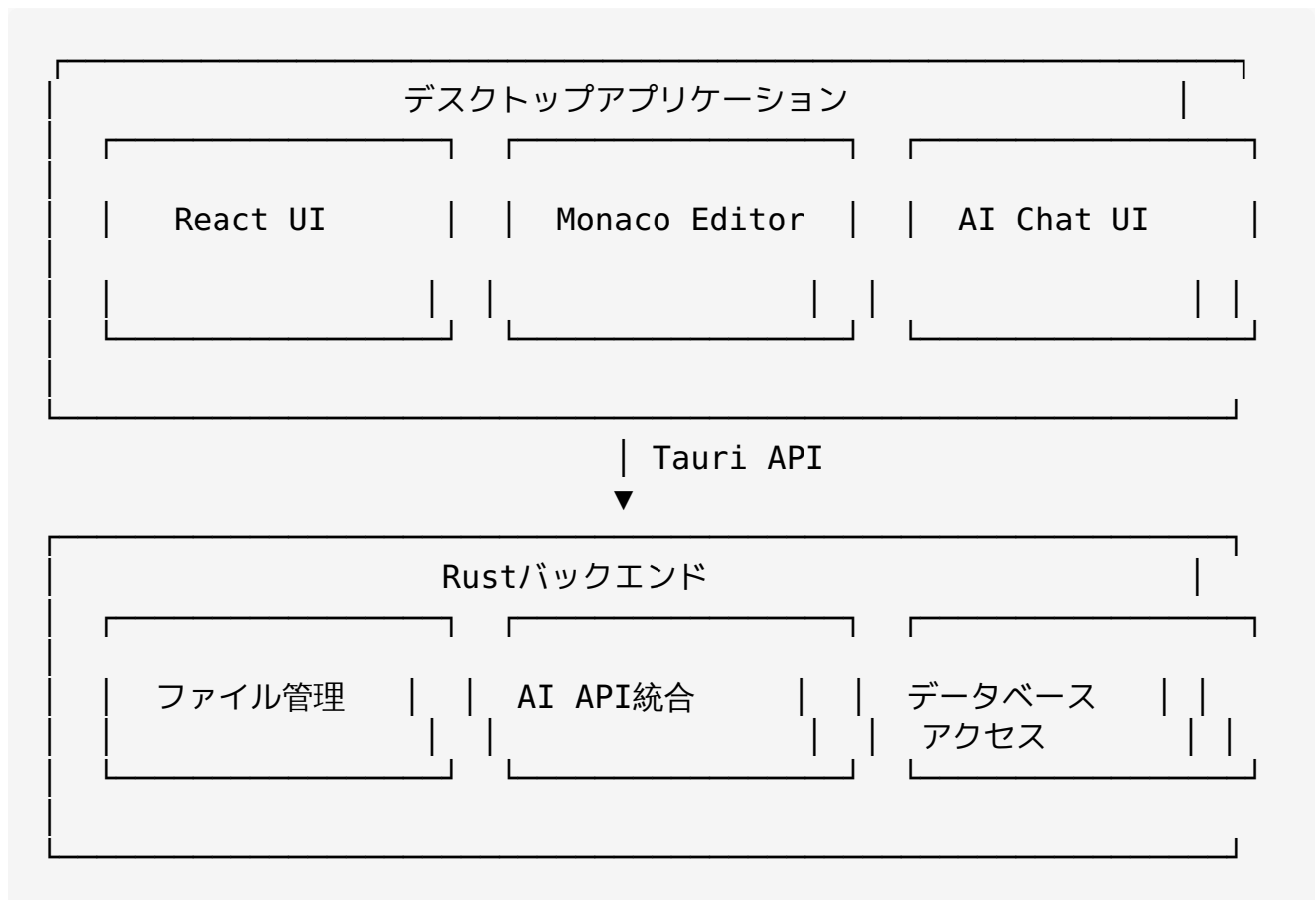
-  AI機能統合済み (Theia AI)

制約

-  EPL 2.0ライセンス (弱いコピーレフト)
-  Theia依存 (プラットフォームロックイン)

4.2 代替案1: Tauri + Monaco Editor

アーキテクチャ



技術構成

- **フレームワーク:** Tauri
- **フロントエンド:** React + TypeScript + Monaco Editor
- **バックエンド:** Rust
- **AI統合:** Rust HTTP Client
- **データベース:** SQLite (ローカル) + PostgreSQL (サーバー)
- **パッケージング:** Tauri Bundle

開発工程

1. **Phase 1 (2-3ヶ月)** : Tauri環境構築・基本UI開発
2. **Phase 2 (3-4ヶ月)** : Monaco Editor統合・エディター機能

- 3. **Phase 3（2-3ヶ月）**：Rust AI統合・RAG機能
- 4. **Phase 4（1ヶ月）**：パッケージング・配布準備

利点

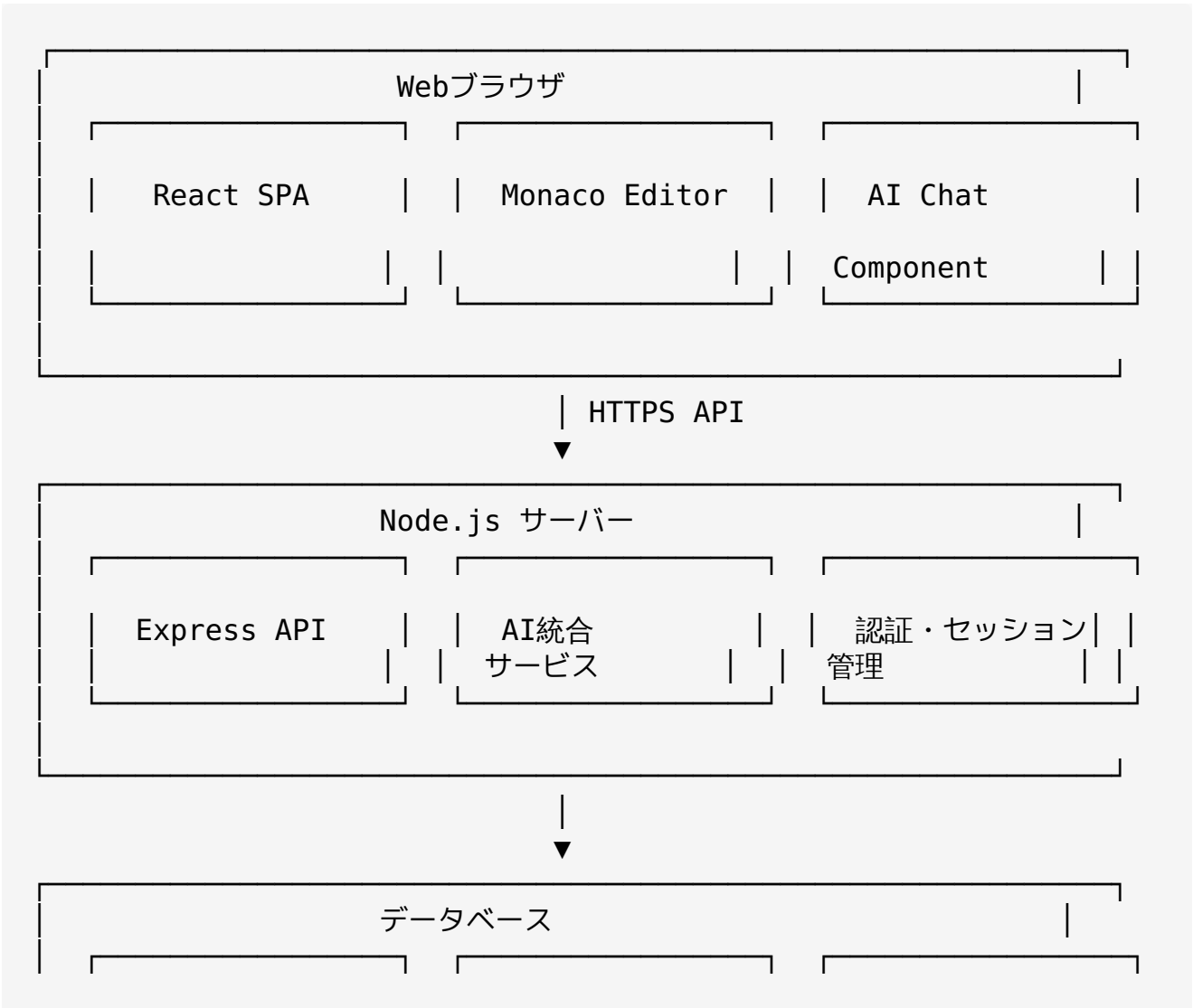
-  軽量・高性能
-  完全独自ブランディング
-  ネイティブアプリ体験
-  クロスプラットフォーム
-  MIT/Apache 2.0ライセンス

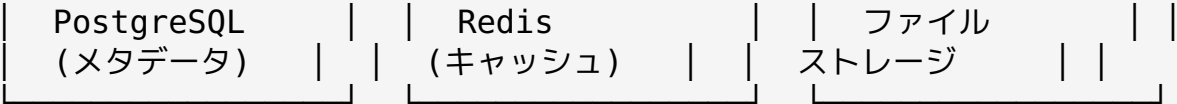
制約

-  Rust学習コスト高
-  開発期間長
-  デバッグ複雑性

4.3 代替案2: Webアプリケーション

アーキテクチャ





技術構成

- ・ フロントエンド: React + TypeScript + Monaco Editor
- ・ バックエンド: Node.js + Express
- ・ データベース: PostgreSQL + Redis
- ・ AI統合: OpenAI/Anthropic SDK
- ・ 認証: JWT + OAuth 2.0
- ・ デプロイ: AWS/Azure/GCP

開発工程

1. **Phase 1 (1-2ヶ月)** : 基本Web UI ・ Monaco統合
2. **Phase 2 (2-3ヶ月)** : バックエンドAPI ・ AI統合
3. **Phase 3 (1-2ヶ月)** : RAG機能 ・ データベース統合
4. **Phase 4 (1ヶ月)** : デプロイ ・ 運用準備

利点

- ・  開発速度最速
- ・  運用コスト低
- ・  スケーラビリティ高
- ・  アクセシビリティ良
- ・  MIT ライセンス

制約

- ・  ネットワーク依存
- ・  ブラウザ制約
- ・  セキュリティ考慮必要

5. ライセンス比較表

技術/ライセンス	商用利用	再配布	改変	サブライセンス	特許権	制約事項
MIT						著作権表示のみ

技術/ライセンス	商用利用	再配布	改変	サブライセンス	特許権	制約事項
Apache 2.0	✓	✓	✓	✓	✓	著作権表示・変更明示・NOTICE保持
BSD 3-Clause	✓	✓	✓	✓	⚠	著作権表示・推奨禁止条項
EPL 2.0	✓	✓	✓	✓	✓	弱いコピーレフト・ソース開示
VSCode製品	⚠	✗	✗	✗	✗	開発・テスト用途のみ
VSCodeソース	✓	✓	✓	✓	⚠	MIT + 商標権制約

ライセンス推奨度ランキング

1. **MIT License** - 最も制約が少なく商用利用に最適
2. **Apache 2.0** - 特許権保護があり企業利用に安心
3. **BSD 3-Clause** - MIT類似だが推奨禁止条項あり
4. **EPL 2.0** - 商用フレンドリーだが弱いコピーレフト

6. 開発コスト・期間見積もり

6.1 開発コスト比較

技術スタック	初期開発	学習コスト	保守コスト	総合評価
Theia IDE	低-中	中	低-中	★★★★★
Tauri + Monaco	中-高	高	中	★★★★★
Webアプリ	低	低	低	★★★★★
完全独自	高	高	高	★★★

6.2 開発期間見積もり

フェーズ	Theia IDE	Tauri + Monaco	Webアプリ	完全独自
設計・環境構築	1ヶ月	2ヶ月	1ヶ月	3ヶ月
コア機能開発	2-3ヶ月	4-5ヶ月	2-3ヶ月	6-8ヶ月
AI統合・RAG	1-2ヶ月	2-3ヶ月	1-2ヶ月	3-4ヶ月
テスト・デプロイ	1ヶ月	1ヶ月	1ヶ月	2-3ヶ月
合計	4-6ヶ月	8-10ヶ月	5-7ヶ月	12-18ヶ月

6.3 リスク評価

技術リスク

- **Theia IDE:** 低（実績あるプラットフォーム）
- **Tauri + Monaco:** 中（Rust学習コスト）
- **Webアプリ:** 低（枯れた技術）
- **完全独自:** 高（未知の技術課題）

ライセンスリスク

- **Theia IDE:** 低（EPL 2.0は商用フレンドリー）
- **Tauri + Monaco:** 最低（MIT/Apache 2.0）
- **Webアプリ:** 最低（MIT）
- **完全独自:** 最低（独自ライセンス）

運用リスク

- **Theia IDE:** 低（コミュニティサポート）
- **Tauri + Monaco:** 中（デスクトップアプリ配布）
- **Webアプリ:** 低（クラウド運用）
- **完全独自:** 高（全て自社対応）

7. 実装ロードマップ

7.1 推奨実装戦略：段階的アプローチ

Stage 1: MVP（Minimum Viable Product） - 3ヶ月

目標: 基本的な約款エディター機能の実現

技術選択: Webアプリケーション（最速実装） - React + Monaco Editor + Node.js - 基本的なAI統合（OpenAI API） - シンプルなテンプレート機能 - ローカルファイル管理

成果物: - 基本エディター機能 - AI対話機能 - 簡単な約款テンプレート - ユーザー認証

Stage 2: 機能拡張 - 3ヶ月

目標: 本格的な約款作成支援機能

機能追加: - RAG機能（参考約款検索） - 高度なテンプレート管理 - バージョン管理 - 複数AIプロバイダー対応 - 承認ワークフロー

Stage 3: プラットフォーム移行 - 6ヶ月

目標: Theia IDEベースへの移行（必要に応じて）

移行理由: - より高度なエディター機能が必要 - デスクトップアプリ化の要求 - VSCode拡張機能の活用

7.2 技術選定フローチャート



7.3 意思決定マトリックス

評価項目	重要度	Theia IDE	Tauri + Monaco	Webアプリ	完全独自
開発速度	高	4	2	5	1
商用利用適性	最高	4	5	5	5
カスタマイズ性	高	3	4	3	5
保守性	高	4	3	4	2

評価項目	重要度	Theia IDE	Tauri + Monaco	Webアプリ	完全独自
コスト効率	高	4	2	5	1
技術リスク	中	4	3	5	2
スケーラビリティ	中	3	2	5	3
ユーザー体験	高	4	5	3	4
総合スコア	-	30	26	35	23

8. 結論と推奨事項

8.1 最終推奨技術スタック

第1推奨：段階的アプローチ

1. **MVP段階:** Webアプリケーション（React + Monaco Editor + Node.js）
2. **本格運用:** 必要に応じてTheia IDEプラットフォームへ移行

理由

-  最速でのMVP実現（3ヶ月）
-  低リスク・低コスト
-  完全な商用利用許可
-  将来的な拡張性確保
-  技術的負債の最小化

8.2 ライセンス戦略

推奨ライセンス体系

- **コアライブラリ:** MIT License（Monaco Editor等）
- **フレームワーク:** MIT/Apache 2.0（React, Node.js等）
- **独自開発部分:** 独自商用ライセンス
- **オープンソース部分:** Apache 2.0（コミュニティ貢献）

ライセンス管理

- 全依存ライブラリのライセンス監査
- 著作権表示の適切な管理
- 商標権侵害の回避

- ・ 定期的なライセンス見直し

8.3 実装優先順位

高優先度（MVP必須）

1. Monaco Editorベースの約款エディター
2. 基本的なAI対話機能
3. シンプルなテンプレート機能
4. ユーザー認証・セッション管理

中優先度（機能拡張）

1. RAG機能（参考約款検索）
2. 高度なテンプレート管理
3. バージョン管理・履歴機能
4. 複数AIプロバイダー対応

低優先度（将来拡張）

1. Theia IDEプラットフォーム移行
2. デスクトップアプリ化
3. モバイル対応
4. 高度なワークフロー機能

8.4 成功要因

技術的成功要因

- ・ 枯れた技術の活用（React, Node.js, Monaco Editor）
- ・ 段階的な機能拡張
- ・ 適切なアーキテクチャ設計
- ・ 継続的なパフォーマンス最適化

ビジネス的成功要因

- ・ 明確なライセンス戦略
- ・ ユーザーフィードバックの積極的活用
- ・ 競合他社との差別化
- ・ 継続的な機能改善

8.5 リスク軽減策

技術リスク

- ・ プロトタイプによる技術検証
- ・ 段階的な実装・テスト
- ・ 複数の技術選択肢の並行検討
- ・ 専門家によるコードレビュー

ライセンスリスク

- ・ 法務専門家による事前確認
- ・ ライセンス管理ツールの導入
- ・ 定期的なライセンス監査
- ・ 商標権侵害の事前調査

ビジネスリスク

- ・ MVP による早期市場検証
- ・ ユーザーニーズの継続的調査
- ・ 競合分析の定期実施
- ・ 柔軟な技術戦略の維持

9. 次のステップ

9.1 即座に実行すべき項目

1. **技術選定の最終決定**（1週間）
2. ステークホルダーとの技術選定会議
3. 開発チームの技術スキル評価
4. 最終的な技術スタック決定
5. **法務確認**（2週間）
6. 選定技術のライセンス法務確認
7. 商標権侵害リスクの評価
8. 独自ライセンス戦略の策定
9. **プロトタイプ開発**（4週間）
10. Monaco Editor統合の技術検証
11. AI API統合の動作確認

9.2 中期的な実行項目

1. **詳細設計**（4-6週間）
2. システムアーキテクチャの詳細設計
3. データベース設計
4. API設計
5. セキュリティ設計
6. **開発環境構築**（2-3週間）
7. CI/CDパイプライン構築
8. 開発・テスト・本番環境準備
9. 監視・ログ基盤構築
10. **MVP開発**（8-12週間）
11. 段階的な機能実装
12. 継続的なテスト・品質確保
13. ユーザーフィードバック収集

9.3 長期的な戦略

1. **市場投入・フィードバック収集**（3-6ヶ月）
2. **機能拡張・プラットフォーム最適化**（6-12ヶ月）
3. **スケールアップ・国際展開**（12ヶ月以降）

レポート作成日: 2025年6月6日

調査期間: 2025年6月6日

調査範囲: VSCode/Clineライセンス、代替技術、商用利用適性

推奨有効期限: 2025年12月（技術動向により見直し推奨）